Predicting Extended Length of Stay in Hospitals

Unison Park & Brianna Gamble

Abstract

Extended hospital length of stay (LOS) is an important quality and efficiency measure in healthcare, closely tied to hospital costs, bed availability, and patient safety. This project compared logistic regression (LR) and random forest (RF) models to predict extended LOS, defined as seven or more days, using Microsoft's publicly available clinical dataset (Microsoft, 2018). LR was chosen as a baseline method due to its interpretability (Cox, 1958), while RF was included to capture nonlinear relationships and interactions between features (Breiman, 2001). Model performance was assessed using precision–recall area under the curve (PR-AUC), receiver operating characteristic area under the curve (ROC-AUC), and calibration metrics (Fawcett, 2006). LR demonstrated strong discrimination (ROC-AUC = 0.95) but struggled with class-level precision and recall. RF achieved superior results (ROC-AUC = 0.99, PR-AUC = 0.94) and provided insights into the most important predictors, including readmission counts, hematocrit, body mass index (BMI), creatinine, sodium, and glucose. These findings highlight the utility of ensemble-based machine learning models for anticipating prolonged hospitalizations and guiding resource allocation.

I. Introduction

Hospital length of stay (LOS) is a key measure of patient care and healthcare efficiency. Prolonged hospitalizations are associated with higher costs, an increased risk of complications, and strain on hospital resources. Identifying patients likely to remain hospitalized for extended periods can improve capacity planning, discharge coordination, and overall quality of care.

Predictive modeling has been used to address this challenge. Traditional methods such as logistic regression provide interpretable results but may be limited when relationships among variables are nonlinear or complex. Ensemble-based machine learning methods, including random forest, are better equipped to capture such interactions and often achieve higher predictive accuracy, particularly in imbalanced datasets where extended LOS cases are less common.

The Microsoft Hospital LOS dataset provides a large, anonymized sample of patient admissions with demographic data, comorbidities, and laboratory values. This project used the dataset to compare logistic regression and random forest models for predicting extended LOS, defined as seven or more days. The goals were to evaluate model performance and identify which variables most strongly influenced prolonged hospital stays.

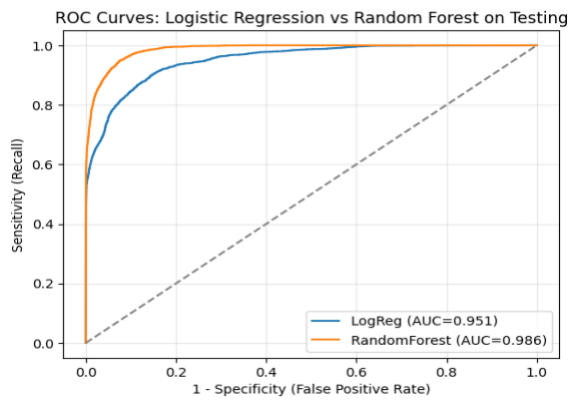
II. Results

Both LR and RF models demonstrated moderate discriminatory performance in predicting extended LOS. LR achieved a cross-validated PR-AUC of 0.86 and a ROC-AUC of 0.95 (Figure 1a). For the non-extended LOS class, precision was 1.00, but recall was limited (0.41). In contrast, the extended LOS class achieved very high recall (0.99) but moderate precision (0.24). The overall accuracy was 0.50, with an F1 score of 0.48.

The RF classifier outperformed the LR across all evaluation metrics in both the training and testing cohorts. RF achieved a PR-AUC of 0.94 and a ROC_AUC of 0.99. For the extended LOS class, recall remained perfect at 1.00, and precision increased to 0.39. For the non-extended LOS class, recall was 0.70, while precision was 1.00. The overall accuracy reached 0.75, with an F1 score of 0.69.

Feature importance analysis from the RF model highlighted readmission counts as the most influential predictor, followed by hematocrit, BMI, creatinine, sodium, glucose, and respiration (Figure 2). These results suggest that both prior utilization patterns and laboratory values are critical in identifying patients at risk of extended LOS.

a.



b.

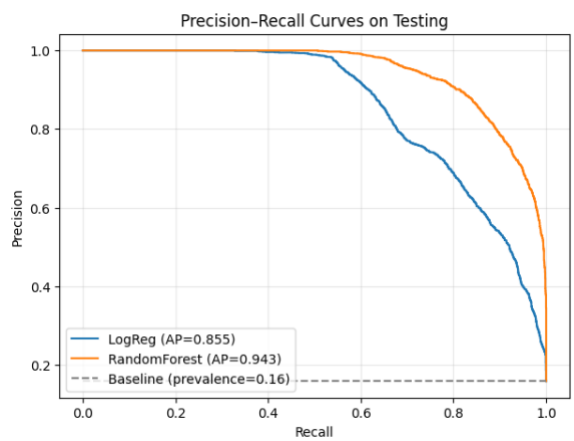


Figure 1. Predictive accuracy for LOS in the test cohort. (a) ROC curves comparing LR and RF. (b) Precision-recall curves comparing LR and RF with average precision values and baseline prevalence.

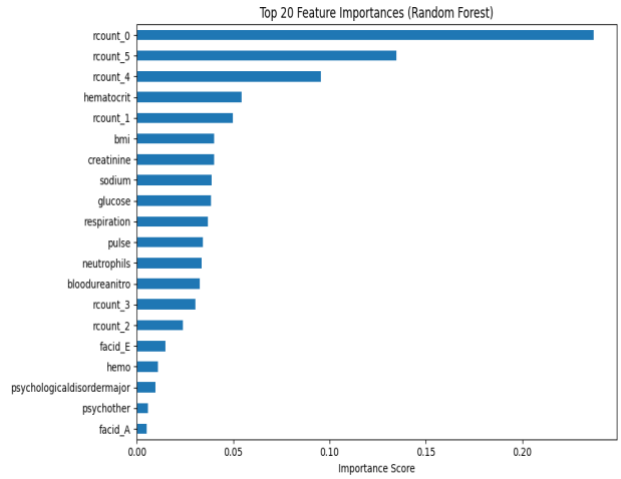


Figure 2. Top 20 features importances from the RF classifier.

III. Discussion

In this study, both LR and RF achieved high discrimination, though RF provided higher performance across multiple evaluation metrics (Figure 1a). While LR achieved high ROC-AUC values, its performance revealed important trade-offs: recall for extended LOS patients was nearly perfect, but precision was low, resulting in a high rate of false positives. This suggests that LR, although interpretable, may be less reliable for clinical decision-making where accurate identification is crucial. In contrast, the RF model achieved a better balance between sensitivity and specificity (Figure 1a). It not only maintained perfect recall for extended LOS cases but also improved precision compared to LR, reducing the amount of false positives. Additionally, RF delivered higher overall accuracy and F1 scores, highlighting its ability to handle the nonlinear relationships and feature interactions present in the dataset. These results reinforce the value of ensemble machine learning methods in healthcare prediction tasks, particularly when dealing with imbalanced data such as extended LOS (Figure 3).

The feature importance analysis further emphasized the strengths of RF.

Readmission counts emerged as the influential predictor, followed by laboratory values and physiological measures (Figure 2). *rcount_0* (no prior readmissions) ranked highest in feature importance mainly because it represented the majority of patients and served as a baseline split for the model. Its prominence reflects statistical leverage in partitioning the data, not that readmissions increase risk. Higher readmission counts remain clinically relevant predictors of extended LOS, but their smaller sample size reduced their relative importance. These findings align with clinical expectations, as readmission and abnormal laboratory values are often markers of underlying illness severity and resource utilization (Hirani et al., 2025).

Despite the results, several limitations existed. First, the dataset was derived from a single public source, which may limit generalizability to other hospital systems or patient populations. Secondly, although preprocessing steps reduce bias, the exclusion of specific variables and the reliance on one-time admission data may omit important contextual factors such as health histories or treatment interventions. Thirdly, the dataset was divided into an 80/20 split for training and testing, but the training set was not further partitioned for validation or model tuning. Lastly, while RF improved prediction performance, its interpretability is lower than that of RF, which may pose challenges for clinical adoption.

IV. Methods

Dataset

The dataset was sourced by Microsoft from their Github repository. It includes 100,000 observations across 27 features. The features include an anonymized clinical dataset containing demographics, comorbidities, laboratory results, and vital signs. Seven variables were excluded due to the potential bias and irrelevance to the model. The dataset contains no missing values as well. Prior to model development, variables were visualized to ensure any potential trends that may arise during training.

Preprocessing

All preprocessing, model development, and evaluation were conducted in Python using the scikit-learn package and related libraries. The dataset was randomly divided into training (80%) and test (20%) cohorts, and baseline variables were compared between the two groups (Table 1). Continuous variables were processed according to their distribution: those exhibiting high skewness were log-transformed and then standardized, while those that were approximately normally distributed were only standardized (Figure 4). Categorical variables such as *rcount* and *facid* were one-hot encoded. Imputation was applied despite the absence of missing values, with the median used for continuous variables and the mode for categorical variables.

Variables	Training Cohort (n = 80000)	Test cohort (n = 20000)
Demographic		
Body mass index (BMI)	29.81 [28.46-31.16]	29.79 [28.42-31.16]
Clinical Comorbidities/flags		
Asthma	2830 (3.5)	697 (3.5)
Dialysis renal end stage	2885 (3.6)	757 (3.8)
Iron deficiency	7579 (9.5)	1915 (9.6)
Pneumonia	3186 (4.0)	759 (3.8)
Substance dependency	5052 (6.3)	1254 (6.3)
Major psychological disorder	19144 (23.9)	4760 (23.8)
Depression	4117 (5.1)	1049 (5.2)
Other psychological disorder	3952 (4.9)	987 (4.9)
Fibrosis	377 (0.5)	103 (0.5)
Malnutrition	3917 (4.9)	1031(5.2)
Blood disorder	6424 (8.0)	1576 (7.9)

Laboratory Values		
Hematocrit	11.90 [10.9–12.9]	11.90 [10.9-12.9]
Neutrophils	9.40 [7.7-11.5]	9.40 [7.7-11.5]
Sodium	137.89 [135.88-139.91]	137.87 [135.85-139.91]
Glucose	142.04 [121.60-162.08]	142.30 [121.98-162.51]
Blood urea nitrogen	12.00 [11-14]	12.00 [11-14]
Creatinine	1.10 [0.96-1.23]	1.10 [0.97-1.24]
Vitals		
Pulse	73.00 [66-81]	74.00 [66-81]
Respiration	6.50 [6.5-6.5]	6.50 [6.5-6.5]
Number of readmissions within last 180 days		
0	43948 (54.9)	11083 (55.4)
1	11996 (15.0)	3011 (15.1)
2	8017 (10.0)	1970 (9.8)
3	6516 (8.1)	1531 (7.7)
4	5533 (6.9)	1408 (7.0)
5	3990 (5.0)	997 (5.0)
Facility ID		
A	24047 (30.1)	5988 (29.9)
B	24002 (30.0)	6010 (30.1)
C	3722 (4.7)	977 (4.9)
D	3633 (4.5)	866 (4.3)
E	24596 (30.7)	6159 (30.8)

Table 1. Baseline characteristics of patients in the training and test cohorts. Median and IQR are shown for the continuous variables, and exact numbers (%) for categorical variables.

Outcome

The primary outcome was defined as extended LOS, with patients classified into two groups: extended LOS (>= 7 days) and non-extended (<7days). The overall distribution of LOS is shown in Figure 3, highlighting that most patients stayed fewer than 7 days, while a smaller group experienced extended LOS.

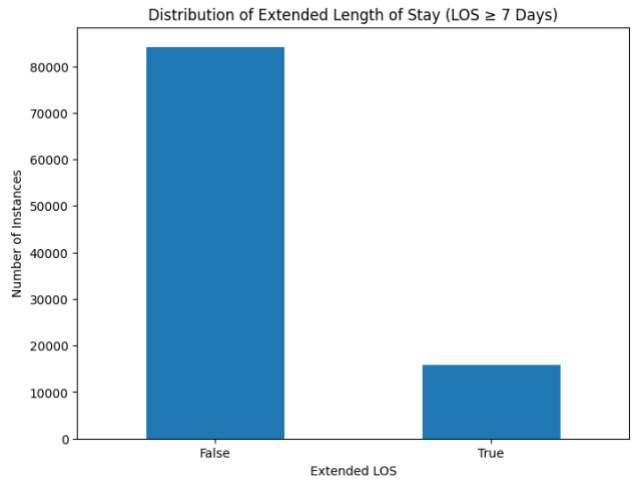


Figure 3. Counts of encounters with and without extended LOS.

Model Development

Two models were constructed within the preprocessing pipelines: logistic regression (LR) and random forest (RF). The LR was employed as a baseline classifier, given its interpretability. Hyperparameter tuning was performed through grid search, varying the regularization penalty (L1 versus L2) and the inverse regularization strength (C) to optimize model performance. An RF classifier was constructed to capture nonlinear feature interaction and complex variable relationships that LR may not adequately model. Hyperparameters for RF, including the number of estimators and the maximum tree features considered at each split, were optimized using randomized search. Both models were embedded within preprocessing pipelines to ensure that transformations such as scaling of continuous variables and encoding of categorical variables were applied consistently. To address class imbalance in the extended LOS outcome, class weight adjustments were applied during model training. All models were trained and evaluated using stratified five-fold cross-validation to preserve the proportion of extended and non-extended LOS cases across folds.

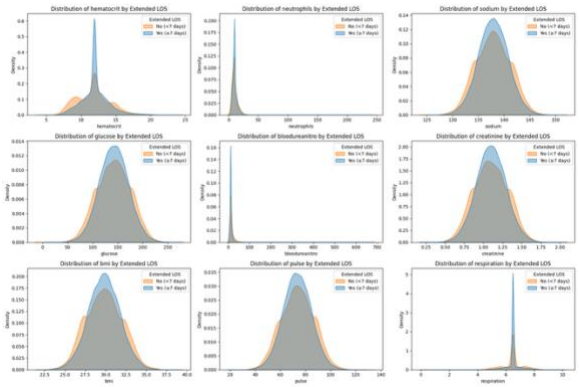


Figure 4. Visualization of continuous variables on extended LOS.

Model Evaluation

Model evaluation focused on discrimination. The primary metric was the area under the precision-recall curve (PR-AUC), chosen for its sensitivity to class imbalance and clinical relevance in identifying tree extended LOS cases. The secondary metric was the area under the receiver operating characteristic curve (ROC-AUC), which reflects the overall discriminatory ability of the models. Additionally, the Brier score was calculated to assess calibration, measuring the accuracy of predicted probabilities in relation to observed outcomes.

A comparison between LR and RF models was made based on the cross-validation mean PR-AUC and ROC-AUC values. Feature importance in the RF model was extracted to provide insight into the relative contribution of variables to model performance. This analysis was intended to complement model performance metrics by identifying predictors most strongly associated with extended hospitalization.

V. Conclusion

This study compared logistic regression and random forest models for predicting extended hospital stays. While both approaches achieved intense overall discrimination, the random forest consistently outperformed logistic regression across multiple metrics, demonstrating higher ROC-AUC and PR-AUC values, as well as a better balance between sensitivity and precision. Analysis of feature importance revealed that prior readmission counts and laboratory values, including hematocrit, creatinine, glucose, and BMI, were among the strongest predictors of extended LOS.

These findings suggest that machine learning models can provide meaningful support in identifying patients at risk for more extended hospitalizations, which may help hospitals manage resources more efficiently and improve patient outcomes. Future research should validate these models in different healthcare systems and expand the feature set to include longitudinal data, procedures, and medications.

AI Use Disclosure

The coding portion of this project utilized assistance from ChatGPT, OpenAI (refer to Supplemental).

References

Breiman, L. (2001). Random forests. *Machine Learning, 45*(1), 5–32. https://doi.org/10.1023/A:1010933404324

Cox, D. R. (1958). The regression analysis of binary sequences (with discussion). *Journal of the Royal Statistical Society: Series B (Methodological), 20*(2), 215–242. https://doi.org/10.1111/j.2517-6161.1958.tb00292.x

Friedman, J. H. (2001). Greedy function approximation: A gradient boosting machine. *Annals of Statistics, 29*(5), 1189–1232. https://doi.org/10.1214/aos/1013203451

OpenAI. (2025). ChatGPT (Sept 27 version) [Large language model]. https://chat.openai.com/

Hirani, R., Podder, D., Stala, O., Mohebpour, R., Tiwari, R. K., & Etienne, M. (2025). Strategies to Reduce Hospital Length of Stay: Evidence and Challenges. *Medicina (Kaunas, Lithuania)*, *61*(5), 922. https://doi.org/10.3390/medicina61050922

Microsoft. (2018). *r-server-hospital-length-of-stay: Data* [Repository]. GitHub. https://github.com/microsoft/r-server-hospital-length-of-stay/tree/master/Data

Supplemental

ChatGPT prompt: Create a simple template python code of machine learning pipeline algorithms, logistic regression and random forest.

Answer:

```
# ==============================================
# Machine Learning Pipeline Template
# Logistic Regression & Random Forest
# ==============================================

# Import libraries
import pandas as pd
from sklearn.model_selection import train_test_split, GridSearchCV
from sklearn.preprocessing import StandardScaler
from sklearn.pipeline import Pipeline
from sklearn.linear_model import LogisticRegression
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import classification_report, accuracy_score

# ==============================================
# Load your dataset
# ==============================================
# Example: replace with your own dataset
# df = pd.read_csv("your_data.csv")

# Dummy dataset for template
from sklearn.datasets import load_breast_cancer
data = load_breast_cancer()
X, y = data.data, data.target

# Train-test split
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y
)

# ==============================================
# Logistic Regression Pipeline
# ==============================================
log_reg_pipeline = Pipeline([
    ("scaler", StandardScaler()),   # Scale features
    ("clf", LogisticRegression(max_iter=1000, random_state=42))
])

log_reg_pipeline.fit(X_train, y_train)
y_pred_lr = log_reg_pipeline.predict(X_test)

print("=== Logistic Regression Results ===")
```

```
print("Accuracy:", accuracy_score(y_test, y_pred_lr))
print(classification_report(y_test, y_pred_lr))

# ==========================================
# Random Forest Pipeline
# ==========================================
rf_pipeline = Pipeline([
    ("clf", RandomForestClassifier(random_state=42))
])

# Example: optional grid search for tuning
param_grid = {
    "clf__n_estimators": [100, 200],
    "clf__max_depth": [None, 5, 10],
}
grid_search = GridSearchCV(rf_pipeline, param_grid, cv=5, scoring="accuracy")
grid_search.fit(X_train, y_train)

best_rf = grid_search.best_estimator_
y_pred_rf = best_rf.predict(X_test)

print("\n=== Random Forest Results ===")
print("Best Params:", grid_search.best_params_)
print("Accuracy:", accuracy_score(y_test, y_pred_rf))
print(classification_report(y_test, y_pred_rf))
```

How the answer was adopted: This response served as an initial step in understanding the expectations for coding in the project. The temple was then adapted to the project's specific needs and refined further. For example, replacing grid search with randomized search to address the long processing time for the RF classifier.

ChatGPT prompt reply: Create a template code for plotting the ROC-AUC and PR-SUC of the test set result comparing logistic regression and random forest.
Answer:

```
# ==== Setup ====
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

from sklearn.datasets import make_classification
from sklearn.model_selection import train_test_split
from sklearn.metrics import (
    roc_curve, roc_auc_score,
    precision_recall_curve, average_precision_score
)
from sklearn.linear_model import LogisticRegression
```

```
from sklearn.ensemble import RandomForestClassifier
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import StandardScaler

# ------------------------------------------------------------
# (A) Demo data — replace with your own X_train, X_test, y_train, y_test if available
# ------------------------------------------------------------
X, y = make_classification(
    n_samples=3000, n_features=20, n_informative=8, n_redundant=5,
    class_sep=1.0, weights=[0.7, 0.3], random_state=42
)
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.25, stratify=y, random_state=42
)

# ------------------------------------------------------------
# (B) Models — feel free to swap in tuned hyperparameters
# ------------------------------------------------------------
log_clf = Pipeline([
    ("scaler", StandardScaler()),
    ("logreg", LogisticRegression(max_iter=500, n_jobs=None, solver="lbfgs"))
])

rf_clf = RandomForestClassifier(
    n_estimators=300, max_depth=None, random_state=42, n_jobs=-1
)

# Train
log_clf.fit(X_train, y_train)
rf_clf.fit(X_train, y_train)

# ------------------------------------------------------------
# Helpers to plot ROC and PR curves on the TEST set
# ------------------------------------------------------------
def plot_roc_on_test(models, model_names, X_test, y_test, title="ROC Curves (Test Set)"):
    plt.figure()
    for model, name in zip(models, model_names):
        # Probability of positive class
        y_proba = model.predict_proba(X_test)[:, 1]
        fpr, tpr, _ = roc_curve(y_test, y_proba)
        auc_val = roc_auc_score(y_test, y_proba)
        plt.plot(fpr, tpr, label=f"{name} (AUC={auc_val:.3f})")

    # chance line
    plt.plot([0, 1], [0, 1], linestyle="dashed", linewidth=1)
    plt.xlabel("False Positive Rate (1 - Specificity)")
```

```
    plt.ylabel("True Positive Rate (Sensitivity/Recall)")
    plt.title(title)
    plt.legend(loc="lower right")
    plt.grid(True, alpha=0.3)
    plt.tight_layout()
    plt.show()

def plot_pr_on_test(models, model_names, X_test, y_test, title="Precision-Recall Curves (Test
Set)"):
    plt.figure()
    for model, name in zip(models, model_names):
        y_proba = model.predict_proba(X_test)[:, 1]
        prec, rec, _ = precision_recall_curve(y_test, y_proba)
        ap = average_precision_score(y_test, y_proba)
        plt.plot(rec, prec, label=f"{name} (AP={ap:.3f})")

    # baseline: positive rate
    pos_rate = np.mean(y_test)
    plt.hlines(pos_rate, 0, 1, linestyles="dashed", linewidth=1)
    plt.xlabel("Recall")
    plt.ylabel("Precision")
    plt.title(title)
    plt.legend(loc="lower left")
    plt.grid(True, alpha=0.3)
    plt.tight_layout()
    plt.show()

# ------------------------------------------------------------
# Compute test-set AUC metrics + plots
# ------------------------------------------------------------
models = [log_clf, rf_clf]
model_names = ["Logistic Regression", "Random Forest"]

# Plots
plot_roc_on_test(models, model_names, X_test, y_test)
plot_pr_on_test(models, model_names, X_test, y_test)

# Summary table of AUCs on TEST set
records = []
for model, name in zip(models, model_names):
    y_proba = model.predict_proba(X_test)[:, 1]
    roc_auc = roc_auc_score(y_test, y_proba)
    pr_auc = average_precision_score(y_test, y_proba)  # AP = PR-AUC
    records.append({"Model": name, "ROC-AUC": roc_auc, "PR-AUC": pr_auc})

summary = pd.DataFrame(records).sort_values("ROC-AUC", ascending=False)
```

```
print(summary.to_string(index=False))
```

How the answer was adopted: Replaced some parts of the coding to align with the project and copied the final results using the training set data.